



DAY 23: PROMISE CHAINING IN JAVASCRIPT



Lakshmiprasanna Vakada

Promises are a powerful way to handle asynchronous operations, but what if we need to execute multiple async tasks one after another? 🤔

✅ This is where Promise Chaining comes in!

◆ What is Promise Chaining?

Promise Chaining allows us to execute a series of asynchronous operations in sequence. Each `.then()` returns a new promise, allowing us to chain multiple `.then()` calls.

◆ Example: Without Chaining (Callback Hell)

Before promises, we used nested callbacks, leading to callback hell:

```
fetchUserData(1, (user) => {  
  fetchOrders(user.id, (orders) => {  
    fetchOrderDetails(orders[0],  
(details) => {  
      console.log(details);  
    });  
  });  
});
```

👉 This structure is hard to read, debug, and maintain.

◆ Example: Using Promise Chaining (Cleaner Code)

Instead of nesting callbacks, we can chain promises like this:

```
fetchUserData(1)
  .then((user) => fetchOrders(user.id))
// Pass the result to the next then
  .then((orders) =>
    fetchOrderDetails(orders[0]))
  .then((details) =>
    console.log(details)) // Final result
  .catch((error) =>
    console.error("Error:", error)); //
Handle errors once
```

✓ Much cleaner & readable!

◆ Breaking it Down Step-by-Step 🔍

Let's assume we have these functions returning Promises:

```
function fetchUserData(userId) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve({ id: userId, name:  
"Alice" }), 1000);  
  });  
}  
  
function fetchOrders(userId) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve(["order1", "order2"]),  
1000);  
  });  
}  
  
function fetchOrderDetails(orderId) {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve({ orderId, item: "Laptop"  
}), 1000);  
  });  
}
```

Now, chaining them together:

```
fetchUserData(1)
  .then((user) => {
    console.log("User:", user);
    return fetchOrders(user.id);
  })
  .then((orders) => {
    console.log("Orders:", orders);
    return fetchOrderDetails(orders[0]);
  })
  .then((details) => console.log("Order
Details:", details))
  .catch((error) => console.error("Error:",
error));
```

◆ Execution Flow:

- 1 `fetchUserData(1)` → Returns user data after 1s
- 2 `fetchOrders(user.id)` → Returns order list after 1s
- 3 `fetchOrderDetails(orders[0])` → Returns order details after 1s

◆ Handling Errors in Promise Chains ❌

Instead of adding `.catch()` at every step, we handle errors once at the end:

```
fetchUserData(1)
  .then((user) => fetchOrders(user.id))
  .then((orders) =>
    fetchOrderDetails(orders[0]))
  .then((details) =>
    console.log(details))
  .catch((error) => console.error("Error
occurred:", error));
```


✓ If any step fails, the error is caught in `.catch()`.

◆ Returning Values in Promise Chains

Each `.then()` should return a value or another Promise:

```
fetchUserData(1)
  .then((user) => {
    console.log(user.name);
    return "Simple Value"; // Returns value
                             to next then
  })
  .then((value) => console.log("Received:",
value))
  .catch((error) => console.error("Error:",
error));
```


◆ Output:

Alice

Received: Simple Value

◆ Converting Callback Hell to Promise Chaining

Before (Callback Hell 😵)

```
doSomething(function (result) {  
  doSomethingElse(result, function  
(newResult) {  
    doThirdThing(newResult, function  
(finalResult) {  
      console.log(finalResult);  
    });  
  });  
});
```

👉 Hard to maintain!

After (Promise Chaining 🚀)

```
doSomething()  
  .then((result) =>  
doSomethingElse(result))  
  .then((newResult) =>  
doThirdThing(newResult))  
  .then((finalResult) =>  
console.log(finalResult))  
  .catch((error) =>  
console.error(error));
```

✅ Easier to read & maintain!